Assignment 3 Report

Author: Luka Karanovic

Date: October 28, 2026

Overview

This report outlines my process of making a Naive Bayes classifier predictive model for a modified version of the German Creditor Dataset. The German Creditor dataset contains a set of bank customers described by a set of attributes, and is generally used to classify these customers as either good or bad credit risks. The data set is downloaded from the OpenML website, and in the original format provided by Prof. Hofmann. The modified version of the data set removed most descriptive attributes and only has the following: "purpose", "employment", "housing", "job", "foreign_worker". The Naive Bayes classifier uses these five descriptive attributes to build a decision tree to predict the ‘Credit_Risk’ target attribute.

The report goes over the suitability of the Naive Bayes classifier model for this application field, the construction of the model, testing process of the model, an evaluation of the model, and a comparison with the decision tree model built on the same German Creditor dataset in assignment 2.

Suitability of Naive Bayes Classifier

I do not think the Naive Bayes classifier is suitable for this application field. For the Naive Bayes classifier to be suitable, the dataset must satisfy the naive big assumption. The assumption is that each descriptive attribute is conditionally independent from one another. This is because the Naive Bayes classifier calculates the probabilities of each attribute in isolation from one another.

As to why the dataset and application field do not satisfy this condition, take the job and foreign worker features for example. An applicant is much more likely to be in the ‘non-resident’ related categories if they are a ‘foreign worker’, this means that the ‘job’ and ‘foreign worker’ fields are not conditionally independent one another.

More examples can be found within the modified training dataset, and tons more in the original dataset (purpose affecting credit amount, age affecting purpose).

While using Naive Bayes often violates the naive big assumption on real-world datasets, it can still sometimes achieve decent classification accuracy. This means it is not the worst choice for this application field despite it not theoretically being applicable to this dataset.

Naive Bayes Classifier Construction Process

The algorithm I ended up using is `Scikit-learn`'s `CategoricalNB` (categorical Naive Bayes classifier) algorithm. The algorithm computes probabilities as follows:

1. CategoricalNB expects integer inputs (0, 1, 2...) and will throw a `ValueError` if it receives a string (e.g. category name). To prevent this, we use `Scikit-learn`'s `OrdinalEncoder` to map categories in our features to integers.
    a. The model internally builds a categorical index map for every column so it knows how many unique categories (n_i) exist per feature.
2. The algorithm computes initial conditional probabilities for each feature based on if the Credit_Risk has value 'bad' or 'good'
    a. For descriptive feature X and category x in that feature, the algorithm aggregates and counts how many times that category appears within each class divided by the total number of items in that class:
        - It gets the actual frequencies of data objects by **counting occurrences** in the training dataset
        - $P(X = x \mid Credit_Risk = bad) = \frac{count(X=x,\ Credit_Risk=bad)}{count(Credit_Risk=bad)}$
        - $P(X = x \mid Credit_Risk = good) = \frac{count(X=x,\ Credit_Risk=good)}{count(Credit_Risk=good)}$
3. It applies the Laplace smoothing formula with $k = 1$ to avoid the pitfall where a probability of 0 wipes out other calculations.
        - $P(X = x \mid Credit_Risk = bad) = \frac{count(X=x,\ Credit_Risk=good) + k}{count(Credit_Risk=good) + k|X|}$
4. `CategoricalNB` immediately wraps decimal probabilities in natural logarithms (ln) because multiplying many tiny probabilities together can cause computer floating-point numbers to "underflow" (crash to zero). Adding logarithms avoids this.
    a. Important: Probabilities shown below are converted from log-space back to their normal decimal values between 0 and 1 (so they are more easily interpretable by humans).

Example Probability Calculation

For example (fake data), say we have 500 applicants with Credit_Risk = 'bad' and 50 applicants with their housing attribute equal to 'rent' that also have a bad credit risk. Then this probability is computed:

$$P(housing = 'rent' \mid Credit_Risk = bad) = \frac{count(housing = 'rent',\ Credit_Risk=bad) + 1}{count(Credit_Risk=bad) + 1|X|} = \frac{50 + 1}{500 + 1(3)} = \frac{51}{503} = 0.1014$$

- Since X is housing, |X| = 3 as housing has 3 possible values.

The above probability is what is shown in the following section, but `CategoricalNB` uses log likelihoods, so we apply step 4:

$$ln(P(housing = 'rent' \mid Credit_Risk = bad)) = ln(0.1014) = -2.289$$

This is what is stored in the model's probability tables.

Probabilities

All probabilities are in decimal format and were rounded to 3 decimal places.

purpose

Category	Probability Given Credit_Risk = 'bad'	Probability Given Credit_Risk = 'good'
Domestic Appliance	0.016	0.012
New Car	**0.289**	0.200
Used Car	0.052	0.123
Business	0.108	0.091
Education	0.084	0.046
Furniture / Equipment	0.185	0.175
Other	0.020	0.012
Radio / TV	0.205	**0.305**
Repairs	0.032	0.023
Retraining	0.008	0.014

---

employment

Category	Probability Given Credit_Risk = 'bad'	Probability Given Credit_Risk = 'good'
Unemployed	0.082	0.053
<1 year	0.225	0.141
1 <= X < 4 years	**0.348**	**0.339**
4 <= X < 7 years	0.127	0.198
>= 7 years	0.217	0.269

---

housing

Category	Probability Given Credit_Risk = 'bad'	Probability Given Credit_Risk = 'good'
For Free	0.153	0.098
Rent	0.227	0.154

Own	**0.620**	**0.748**

---

job

Category	Probability Given Credit_Risk = 'bad'	Probability Given Credit_Risk = 'good'
Unemployed / Unskilled Non-Resident	0.025	0.023
Unskilled Resident	0.193	0.205
Skilled	**0.597**	**0.637**
High Qualification / Self Employed / Management	0.185	0.135

---

foreign_worker

Category	Probability Given Credit_Risk = 'bad'	Probability Given Credit_Risk = 'good'
Yes	0.012	0.046

No	**0.988**	**0.954**

Testing

How Predictions Are Made

Predictions rely on **Bayes' Theorem** combined with the **"naive" big assumption of conditional independence**. The model used (`Scikit-learn`'s `CategoricalNB`) operates differently from the traditional Naive Bayes classifier. This section walks through an example test data item being inputted into the model to show the process of how probabilities are calculated and a prediction is made.

Suppose we receive a single applicant X with these 5 descriptive features and an unknown credit risk:

- purpose: 'new car'
- employment: '1-4 years'
- housing: 'own'
- job: 'skilled'
- foreign_worker: 'yes'

Step 1: Ordinal Encoding

Before CategoricalNB can inspect the object, it cannot accept text strings. The OrdinalEncoder translates each string into an integer index based on the mapping it learned during training:

- purpose: ('new car') → Integer Index 1
- employment: ('1-4 years') → Integer Index 2
- housing: ('own') → Integer Index 0
- job: ('skilled') → Integer Index 2
- foreign_worker: ('yes') → Integer Index 1

Now, the model sees X as an array of integers: [1, 2, 0, 2, 1].

Step 2: Retrieving Log-Priors and Likelihoods

The model uses those integer indices to look up pre-calculated log-probabilities from its tables (`class_log_prior_` and `feature_log_prob_`). These were calculated above when determining the probability tables, but the process will be re-explained here in the context of the model making a prediction.

For our example, let's use log-probability values computed during training:

1. Baseline Class Priors

Take the normal probability of each target attribute class and then compute the natural log of that probability:

- $\ln(P(Credit_Risk = bad)) = -1.2040$ (roughly 30%)
- $\ln(P(Credit_Risk = good)) = -0.3567$ (roughly 70%)

2. Feature Category Log-Likelihoods

For example, let's look at the foreign_worker attribute:

- From above, we know (note that Laplace smoothing has already been applied):
- $P(foreign_worker = yes \mid Credit_Risk = bad) = 0.012$
- $P(foreign_worker = yes \mid Credit_Risk = good) = 0.046$

We then take the natural log of each of these probabilities:

- $ln(P(foreign_worker = yes \mid Credit_Risk = bad)) = ln(0.012) = -4.42$
- $ln(P(foreign_worker = yes \mid Credit_Risk = good)) = ln(0.046) = -3.08$

We do this for each attribute, afterward, we sum up the log likelihoods for conditional probabilities for each target attribute class, plus the probability for that target attribute class:

Ex:

$$Score_{bad} = \ln P(Credit_Risk = bad) + \sum(Feature\ Log\ Likelihoods\ given\ Credit_Risk = bad)$$

Attribute Name	Value	Probability (Credit_Risk = good)	Probability (Credit_Risk = bad)	Log likelihood (Credit_Risk = good)	Log likelihood (Credit_Risk = bad)
purpose	new car	0.200	0.289	-1.609	-1.241
employment	1 <= X < 4 years	0.339	0.348	-1.082	-1.056
housing	own	0.748	0.620	-0.290	-0.478
job	skilled	0.637	0.597	-0.451	-0.516
foreign_worker	yes	0.046	0.012	-3.079	-4.423

Adding up the values from the table (and the original baseline class priors), we get:

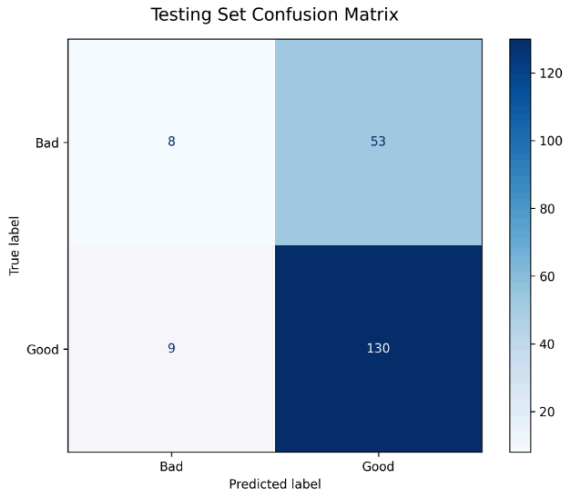
$$Score_{bad} = (-1.2040) + (-1.2400) + (-1.0500) + (-0.3600) + (-0.5100) + (-4.4228)$$
$$Score_{bad} = -8.7868$$

$$Score_{good} = (-0.3567) + (-1.6100) + (-0.9200) + (-0.2200) + (-0.4300) + (-3.0790)$$
$$Score_{good} = -6.6157$$

The model then predicts whatever class produces the greatest (least negative) score, in this case, that the Credit_Risk is ‘good’. This is because a log likelihood closer to 0 is associated with a probability closer to 1.

Comparing Predicted Credit Risks With Reality

All 200 items in the training dataset ran through the above process so the model could predict a Credit_Risk value for each one. The prediction the model made was then compared with the actual Credit_Risk target attribute value, the squares going down diagonally from the top-left are accurate predictions and the other squares are inaccurate predictions.



- As you can see, the model predicted correctly for (130+8)/200 (69%) of objects, and incorrectly for (9+53)/200 (31%) of objects.
- The model was quite bad at predicting when the credit risk was actually ‘Bad’, incorrectly classifying it as ‘Good’ 53 times.

The cost matrix presented in the Excel spreadsheet’s ‘Metadata’ sheet is misaligned from my confusion matrix. Here is the cost matrix after realigning the columns so they match my confusion matrix:

0	5
1	0

Coupling with our confusion matrix, and summing, we get a total error cost (score where lower is better) of the model:

$$0(8) + 5(53) + 1(9) + 0(130) = 0 + 265 + 9 + 0 = 274$$

Naive Bayes Classifier vs. Decision Tree

The testing results of the Naive Bayes classifier and decision tree models are in the following table:

Metric	Decision Tree	Naive Bayes Classifier
Predicted Good, Actual Good	127	130
Predicted Good, Actual Bad	54	53
Predicted Bad, Actual Good	12	9
Predicted Bad, Actual Bad	7	8
Total Error Cost	282	274

The Naive Bayes classifier makes very marginal improvements over the decision tree model, predicting accurately for good credit risks 3 more times than the decision tree and accurately for bad credit risks 1 more time than the decision tree. The total error cost was 8 points lower, which is a 2.84% improvement from the decision tree.

The overall pattern of the predictions were almost identical, meaning either both models are not amazing for this application domain, too few descriptive attributes are used, or the data collected is not good. My guess is that too few descriptive attributes are used, as only 5 of 20 are used, and a lot of the discarded attributes have useful information such as credit amount, age, and credit history. I also think that the data collected is not good. The dataset has many inconsistencies with what is typically expected in the domain so it likely does not represent realistic, natural patterns that models are good at uncovering.

One advantage of using a probability-based model over an information-based model is that the probability-based model gives you more information. Probability-based models not only tell you what the most likely class is, but also how much more likely it is over the other classes. Information-based models do not do this, as it only outputs the class that it believes is most likely and not the computed probabilities or likelihoods for the data object being in each class. Since the performance of the two models is similar, I am more inclined to choose the Naive Bayes classifier over the decision tree to make predicts in this application field.